Master 1 IA-BD — EPL

Cours INF2229 • Informatique et Gestion de Données

Traducteur Multilingue

FR • EN • ÉWÉ

avec reconnaissance et synthèse vocales

Rapport technique

Réalisé par

Groupe GR02

AZIGLOSSOU Judith Gracia
NAPO NISSAO Rafia
WOGLO Koffi Boris Gilbride
PIO Sikirou Afeez Prince Cyrille
KOMBATE Gountande

Encadré par

Dr. Akouyo Yvette GBEDEVI

Année universitaire 2025 – 2026

Résumé

Ce projet réalise un système de traduction automatique **multidirectionnel** entre le français, l'anglais et l'éwé — langue gbe parlée au Togo et au Ghana, fortement sous-dotée en ressources numériques. Le système couvre les **six** directions de traduction possibles et il est complété d'une **chaîne vocale** de bout en bout (reconnaissance et synthèse), l'éwé compris.

L'approche méthodologique privilégie le **transfert d'apprentissage** : plutôt que d'entraîner un modèle depuis zéro, on adapte un grand modèle multilingue pré-entraîné, **NLLB-200** de Meta, à l'aide d'un adaptateur **LoRA** (*Low-Rank Adaptation*). Le modèle est affiné sur un corpus trilingue de près de **700 000 paires** de phrases alignées, nettoyées et filtrées, puis enrichi par **back-translation** pour renforcer les directions les plus pauvres.

L'évaluation, conduite à la fois sur un jeu interne et sur le jeu de référence international **FLORES-200**, montre que l'adaptateur améliore les **six directions sans aucune régression**, avec un gain moyen de **+1,24 point BLEU** et **+1,64 point chrF** par rapport au modèle de base. La solution est exposée via une API FastAPI et une interface web React.

Contribution principale. Un traducteur trilingue opérationnel pour une langue peu dotée, accompagné d'une méthodologie de données rigoureuse (alignement, filtrage, contrôle du biais) et d'une évaluation honnête sur benchmark externe — démontrant un apport mesurable du fine-tuning tout en documentant ses limites.

Sommaire

Résumé **1**

1 Introduction et contexte **4**

1.1 Problématique 4

1.2 Objectifs 4

1.3 Choix de conception 4

2 Architecture générale **6**

3 Construction du corpus **7**

3.1 Sources 7

3.2 Nettoyage et normalisation 7

3.3 Alignement et filtrage du bruit 7

3.4 Tokenisation 8

3.5 Découpage et volumétrie 8

3.6 Contrôle du biais 8

4 Modèle de traduction **9**

4.1 Modèle de base : NLLB-200 9

4.2 Adaptation par LoRA 9

4.3 Dynamique d'entraînement 9

4.4 Augmentation par back-translation 10

5 Évaluation **11**

5.1 Métriques 11

5.2 Protocole 11

5.3 Résultat principal : base vs LoRA sur FLORES-200 11

5.4 Test interne 12

5.5 Analyse 12

6 Chaîne vocale **13**

7 Interface et API **14**

8 Limites et perspectives **15**

8.1 Limites 15

8.2 Perspectives 15

9 Conclusion **16**

1 Introduction et contexte

1.1 Problématique

La traduction automatique a connu des progrès spectaculaires pour les langues dites « à haute ressource » (anglais, français, etc.), mais les centaines de langues africaines restent largement en marge de ces avancées. L'**éwé** (autonyme *Eʋegbe*), parlé par plusieurs millions de locuteurs au Togo et au Ghana, en est un exemple représentatif : il dispose de très peu de corpus numériques, et les rares données disponibles sont souvent bruitées, mal alignées, ou cantonnées à un domaine unique (textes religieux notamment).

Ce projet s'attaque directement à ce déséquilibre en construisant un traducteur couvrant les trois langues du sujet et leurs six directions, avec une attention particulière portée à la **qualité des données** et à une **évaluation rigoureuse** — les deux conditions sans lesquelles un score chiffré n'a pas de sens pour une langue peu dotée.

1.2 Objectifs

Conformément au cahier des charges, le système couvre l'intégralité de la chaîne de traitement :

— **Collecter** des corpus parallèles dans les trois langues à partir de sources ouvertes et de qualité contrôlée.

— **Prétraiter** les données : nettoyage, normalisation Unicode, tokenisation et alignement des phrases.

— **Entraîner** un modèle de traduction neuronale pour les paires Français ↔ Anglais, Français ↔ Éwé et Anglais ↔ Éwé.

— **Intégrer** une reconnaissance vocale (STT) et une synthèse vocale (TTS) pour la traduction orale.

— **Développer** une interface web permettant de tester le traducteur (saisie texte, dictée, écoute, historique).

1.3 Choix de conception

Trois décisions structurent l'ensemble du travail. Premièrement, le recours au **transfert d'apprentissage** depuis un modèle massivement multilingue : c'est la réponse recommandée par la littérature face au manque de données. Deuxièmement, la priorité donnée au **nettoyage et au filtrage** du corpus, car sur une langue peu dotée le bruit pèse davantage qu'un modèle plus gros. Troisièmement, une **évaluation sur bench-**

mark externe (FLORES-200) afin d’obtenir des chiffres comparables et honnêtes, et non flattés par la proximité au domaine d’entraînement.

2 Architecture générale

Le projet est organisé en quatre blocs indépendants et reproductibles, qui se transmettent leurs sorties par fichiers. Cette séparation permet de relancer une étape sans rejouer les précédentes, et de tracer précisément l'origine de chaque résultat.

1. **Construction des données** — `notebooks/00_donnees.ipynb`. Collecte, nettoyage, alignement, filtrage et découpage. Produit le corpus final dans `data/processed/nllb/` (fichiers `nllb_train.csv`, `nllb_valid.csv`, `nllb_test.csv`).
2. **Modèle de traduction** — `notebooks/01_pipeline_nllb_lora.ipynb`. Fine-tuning LoRA de NLLB-200, suivi des pertes, évaluation BLEU/chrF et comparaison base vs adaptateur sur FLORES-200.
3. **Augmentation** — `notebooks/02_backtranslation_nllb_lora.ipynb`. Génération de données synthétiques par back-translation, ré-entraînement dans un dossier séparé et comparaison systématique avant adoption.
4. **Service applicatif** — `src/traducteur/`. API FastAPI (endpoints de traduction, transcription, synthèse, historique) et interface React/Vite.

Pile technologique. Python, PyTorch, Hugging Face Transformers et PEFT pour le modèle ; `sacrebleu` pour l'évaluation ; `librosa`, Whisper et MMS pour l'audio ; FastAPI (backend) et React/Vite (frontend) ; Git/GitHub pour la collaboration. Ces choix correspondent aux technologies recommandées par le sujet.

3 Construction du corpus

La qualité du corpus est le facteur déterminant pour une langue peu dotée. Le notebook de données enchaîne sept étapes, de la collecte au bilan final.

3.1 Sources

Le corpus combine plusieurs jeux ouverts, choisis pour leur licence et leur niveau de qualité, et répartis en deux familles :

— **Sources « propres » (humaines)** : **google/SMOL** (traductions professionnelles en–ee), **FLORES** (découpage `dev`, aligné dans les trois langues), **MAFAND** (fr–ewe, actualités, projet Masakhane). Elles entrent directement dans le corpus.

— **Sources web-minées** : OPUS (NLLB, TED2020, Tatoeba, News-Commentary, translatewiki) et une part biblique. Plus volumineuses mais plus bruitées, elles passent par tout le pipeline de durcissement décrit ci-dessous.

3.2 Nettoyage et normalisation

Chaque phrase est nettoyée (suppression des caractères de contrôle et des artefacts de balisage), puis **normalisée en Unicode** afin d'unifier les multiples représentations possibles des caractères éwé accentués et des signes diacritiques. Cette normalisation est essentielle : sans elle, deux occurrences visuellement identiques d'un même mot éwé peuvent être encodées différemment et fausser à la fois l'apprentissage et l'évaluation.

3.3 Alignement et filtrage du bruit

Les paires web-minées subissent un filtrage en plusieurs passes :

1. **Alignement par similarité cosinus.** Chaque phrase est encodée par l'encodeur de NLLB ; on ne conserve une paire que si la similarité cosinus entre la source et la cible dépasse un seuil. Cela élimine les paires mal appariées.
2. **Identification de langue (LID).** Un filtre fastText (modèle NLLB-LID) écarte les phrases dans la mauvaise langue. La LID seule étant peu fiable sur l'éwé en textes courts, elle est **complétée d'une heuristique de caractères éwé** : on vérifie la présence de lettres propres à la langue (ɖ ƒ ŋ ɔ ɛ ɣ ʋ), ce qui réduit fortement la confusion avec l'anglais.
3. **Déduplication** des paires identiques, puis **durcissement du seuil** cosinus sur les sources les plus bruitées avant la construction finale.

3.4 Tokenisation

La tokenisation n'est pas réimplémentée : elle est déléguée au tokenizer **SentencePiece** de NLLB, un modèle de sous-mots de type *unigram*. Ce choix est cohérent avec le modèle utilisé et présente deux avantages décisifs pour l'éwé : il ne dépend pas du découpage par espaces, et il gère les mots inconnus en les décomposant en sous-unités. La langue source et la langue cible sont injectées directement dans la tokenisation au moyen des codes `fra_Latn`, `eng_Latn` et `ewe_Latn`, ce qui permet au même modèle de traiter les six directions.

3.5 Découpage et volumétrie

Le corpus est découpé en `train`/`valid`/`test` une fois pour toutes, et ce découpage est **réutilisé à l'identique** dans toutes les expériences : aucune phrase du test ou de la validation n'apparaît à l'entraînement, ce qui garantit l'absence de fuite et la comparabilité des scores.

Split	EN→EWE	EN→FR	EWE→EN	EWE→FR	FR→EN	FR→EWE	Total
Train	104 499	115 223	104 499	115 184	115 223	115 276	**669 904**
Validation : 13 853 paires • Test : 13 987 paires							

Table 1 – Volumétrie du corpus trilingue final. Les six directions sont volontairement équilibrées.

3.6 Contrôle du biais

Un risque majeur sur l'éwé est le **biais de domaine biblique** : une grande partie des corpus disponibles provient de traductions religieuses, dont le vocabulaire et le style ne reflètent pas la langue courante. Deux mesures l'atténuent : chaque paire porte un drapeau `is_religious` permettant de quantifier et de plafonner cette part (limitée à **36 530** lignes au train, soit environ **5,5 %**) ; et le corpus **Ghana NLP** (en–ewe, d'origine essentiellement biblique et fortement biaisé) a été **écarté** du jeu final.

4 Modèle de traduction

4.1 Modèle de base : NLLB-200

Le système s'appuie sur `facebook/nllb-200-distilled-600M`, une version distillée (600 millions de paramètres) du modèle *No Language Left Behind* de Meta. C'est un Transformer encodeur–décodeur entraîné sur 200 langues, dont l'éwé. La version distillée offre le meilleur compromis entre qualité et coût : elle tient sur un seul GPU grand public et reste rapide en inférence, ce qui est indispensable pour une application interactive.

4.2 Adaptation par LoRA

Affiner les 600 millions de paramètres serait coûteux et risquerait l'oubli catastrophique des connaissances multilingues du modèle. On emploie donc **LoRA** (*Low-Rank Adaptation*) : on **gèle entièrement** le modèle de base et on n'apprend que de petites matrices de rang faible, insérées dans les couches d'attention et les couches *feed-forward*. Seuls quelques pour cent des paramètres sont entraînés, ce qui réduit massivement la mémoire et le temps de calcul, tout en préservant le socle pré-entraîné. La matrice d'embedding et la tête de sortie restent gelées : la tokenisation et les représentations de NLLB ne sont donc pas dégradées.

Hyperparamètre	Valeur et justification
Modèle de base	`facebook/nllb-200-distilled-600M` (encodeur–décodeur, 200 langues).
Rang LoRA	$r = 32$, $\alpha = 64$, dropout $0{,}05$ — capacité suffisante pour apprendre à générer l'éwé.
Modules adaptés	`q_proj,k_proj,v_proj,out_proj,fc1,fc2` — attention *et* feed-forward.
Époques	2 (soit 6 250 pas) — au-delà, la perte de validation plafonne.
Lot effectif	64 (32 $\times$ 2 accumulations de gradient).
Taux d'apprentissage	2×10^{-4}, avec *warmup* de 3 %.
Précision	bf16 + *gradient checkpointing* pour tenir en mémoire GPU.
Longueur de séquence	128 tokens en source et en cible.
Sélection	*load best model* : on conserve le point de contrôle de meilleure `eval_loss`.

Table 2 – Configuration d'entraînement de l'adaptateur LoRA et justifications.

4.3 Dynamique d'entraînement

La perte de validation décroît de façon **monotone** tout au long des deux époques, sans aucun rebond — signe d'une bonne généralisation et de l'absence de sur-apprentissage.

Elle baissait encore légèrement en fin d'entraînement, ce qui suggère une marge résiduelle (cf. perspectives). Le mécanisme *load best model* garantit que l'on conserve le meilleur état même en cas de léger sur-apprentissage tardif.

Pas	Époque	Perte de validation
1 000	0,32	1,1487
3 000	0,96	1,1273
5 000	1,60	1,1184
6 250	2,00	**1,1165**

Table 3 – Décroissance régulière de la perte de validation (extraits).

4.4 Augmentation par back-translation

Pour renforcer les directions *vers l'éwé*, les plus pauvres en données, on applique la **back-translation**. Le principe : conserver l'**éwé humain en cible** (donc de qualité garantie) et fabriquer une **source synthétique** à l'aide du modèle déjà entraîné, uniquement à partir des paires propres `(ewe,fr)` du train. Trois jeux sont produits :

— **A** (`fr` → `en`) : génère des paires `en_synth` → `ewe` et enrichit ainsi la direction EN→EWE en contenu varié et non biblique.

— **A2** (réutilisation) : le même anglais synthétique sert à créer `ewe` → `en_synth`, ce qui équilibre EWE→EN sans calcul GPU supplémentaire.

— **B** (`ewe` → `fr`) : génère des paires `fr_synth` → `ewe` pour renforcer FR→EWE et réduire l'*off-target* (sortie en mauvaise langue).

Seules les cibles éwé **pristines** (similarité cosinus élevée *et* présence de caractères éwé) alimentent ce processus, afin de ne pas injecter de bruit. Le modèle augmenté est entraîné dans un dossier distinct, puis **comparé** au modèle de référence : il n'est adopté que s'il améliore en↔ewe *sans* régresser ailleurs — la back-translation pouvant dégrader si le synthétique est de mauvaise qualité.

5 Évaluation

5.1 Métriques

Deux métriques complémentaires sont utilisées via la bibliothèque `sacrebleu` :

— **BLEU** mesure le recouvrement de n-grammes de *mots* entre la prédiction et la référence. C'est la métrique historique, comparable à la littérature.

— **chrF** mesure le recouvrement de n-grammes de *caractères*. Elle est particulièrement pertinente pour l'éwé, langue à morphologie riche et à diacritiques, où une métrique au mot pénalise injustement de petites variations.

La génération utilise une recherche en faisceau (*beam search*) à 4 faisceaux.

5.2 Protocole

L'évaluation est conduite sur deux jeux, par direction :

— **Test interne** (`nllb_test.csv`) : issu de la même distribution que l'entraînement. Utile, mais **optimiste**.

— **FLORES-200** (`devtest`) : jeu de référence multilingue de Meta, **jamais vu** à l'entraînement et aligné dans les trois langues. C'est la mesure **honnête** et comparable à l'état de l'art, y compris pour en↔ewe.

5.3 Résultat principal : base vs LoRA sur FLORES-200

La comparaison oppose le NLLB de base (adaptateur désactivé) à notre adaptateur LoRA, sur exactement les mêmes phrases. L'adaptateur **améliore les six directions**.

Direction	BLEU			chrF		
	base	LoRA	gain	base	LoRA	gain
EN→EWE	12,65	13,34	**+0,69**	38,42	39,44	**+1,03**
EN→FR	45,89	47,96	**+2,06**	68,18	69,88	**+1,70**
EWE→EN	15,16	16,05	**+0,89**	37,94	39,86	**+1,92**
EWE→FR	11,45	13,62	**+2,17**	34,36	37,82	**+3,47**
FR→EN	42,76	43,49	**+0,73**	66,12	67,12	**+0,99**
FR→EWE	9,86	10,76	**+0,90**	35,61	36,34	**+0,73**
Moyenne	**22,96**	**24,20**	**+1,24**	**46,77**	**48,41**	**+1,64**

Table 4 – FLORES-200 `devtest` : NLLB de base vs adaptateur LoRA (200 phrases par direction).

5.4 Test interne

	EN→EWE	EN→FR	EWE→EN	EWE→FR	FR→EN	FR→EWE
BLEU	29,68	39,77	34,14	35,98	43,27	29,52
chrF	52,88	63,85	53,31	53,51	64,41	51,41

Table 5 – Scores sur le test interne (distribution proche de l'entraînement).

5.5 Analyse

— **Aucune régression.** Sur le benchmark externe, le fine-tuning aide les six directions à la fois (+1,24 BLEU et +1,64 chrF en moyenne). C'est le résultat le plus solide et le plus défendable du projet.

— **Écart test interne / FLORES.** FR→EWE atteint 29,5 BLEU en interne mais seulement 10,8 sur FLORES. Cet écart, attendu, illustre pourquoi un test proche du domaine d'entraînement est trompeur : seul FLORES donne une mesure réaliste de la généralisation.

— **Génération vers l'éwé.** Les directions →EWE progressent le moins (+0,69 et +0,90 BLEU) et restent basses en valeur absolue. C'est la conséquence directe de la rareté des cibles éwé de qualité, et le principal axe d'amélioration.

Exemple qualitatif (EN→EWE).
Source : *« Vatican City's population is around 800. »*
Sortie du modèle : *« Vatika dukɔ fe agbɔsɔsɔ nye 800. »*
La sortie est une phrase éwé bien formée, employant correctement les caractères spécifiques de la langue.

6 Chaîne vocale

La chaîne vocale couvre les trois langues sans dépendance à un outil externe comme `ffmpeg` : le navigateur enregistre directement en WAV, décodé côté serveur par `librosa` (rééchantillonnage à 16 kHz, mono).

— **Reconnaissance vocale (STT). Whisper** (OpenAI) pour le français et l'anglais ; **MMS-ASR** (Meta, fondé sur Wav2Vec2 et couvrant plus de 1 100 langues) pour l'**éwé**, via son adaptateur de langue dédié.

— **Synthèse vocale (TTS). MMS-TTS** (Meta) pour l'éwé, que les navigateurs ne savent pas prononcer ; la voix native du navigateur pour le français et l'anglais.

Optimisations de performance. L'inférence audio s'exécute sur GPU détecté automatiquement ; les modèles sont **préchargés au démarrage** du serveur (ce qui élimine la latence du premier appel) ; l'adaptateur de langue MMS est mis en cache pour ne pas être rechargé à chaque requête ; et la transcription s'exécute hors de la boucle asynchrone, afin de ne pas bloquer le service.

7 Interface et API

Le backend FastAPI expose des endpoints clairs : `/translate` (traduction texte), `/transcribe` (reconnaissance vocale), `/synthesize` (synthèse vocale) et `/history` (historique). Le modèle de traduction et les modèles audio y sont chargés une seule fois et réutilisés.

L'interface React/Vite offre les fonctionnalités demandées par le cahier des charges :

— **Saisie texte** avec traduction quasi instantanée (déclenchée après une courte pause de frappe).

— **Dictée vocale** : enregistrement puis transcription côté serveur, unifiée pour les trois langues, avec un indicateur visuel de l'état (enregistrement, puis transcription en cours).

— **Écoute** de la traduction par synthèse vocale.

— **Historique** des traductions, synchronisé côté serveur entre navigateurs (authentification Google optionnelle).

8 Limites et perspectives

8.1 Limites

— **Génération vers l'éwé** : c'est le maillon faible (BLEU FLORES de l'ordre de 10 à 13). La disponibilité de cibles éwé de qualité reste le facteur limitant.

— **Hétérogénéité des données** : malgré le filtrage, une part de contenu biblique subsiste et le style global demeure inégal.

— **Couverture audio de l'éwé** : la qualité du STT/TTS éwé dépend des modèles MMS pré-entraînés, non affinés sur des locuteurs spécifiques.

8.2 Perspectives

— Prolonger l'entraînement (la perte de validation baissait encore en fin de course).

— Étendre la back-translation EN→EWE à plus grande échelle.

— Augmenter le rang LoRA sur le décodeur pour accroître la capacité de génération.

— Collaborer avec des locuteurs natifs pour annoter du contenu courant — le levier le plus efficace à long terme pour une langue peu dotée.

9 Conclusion

Le projet livre un traducteur **trilingue opérationnel** sur les six directions, fondé sur le fine-tuning LoRA de NLLB-200, un corpus de près de 700 000 paires soigneusement filtrées et une augmentation par back-translation. Sur le benchmark externe FLORES-200, l'adaptateur **surpasse le modèle de base dans toutes les directions** (+1,24 BLEU, +1,64 chrF en moyenne), sans aucune régression. La chaîne vocale — éwé compris — et l'interface web complètent une solution de bout en bout, conforme aux livrables attendus.

La principale marge de progrès demeure la génération vers l'éwé, contrainte par la disponibilité des données : un défi inhérent aux langues peu dotées, que ce travail documente, mesure honnêtement et adresse explicitement par sa méthodologie de données et d'augmentation.

Livrables. Jeu de données trilingue (notebook 00) • modèles entraînés, traduction et audio (`models/`) • code source : notebooks, API et interface • ce rapport technique • démonstration via l'interface web.